Explainable Medical Question Answering using Multi-Agent RAG Systems

AKHIL SAINI, Otto von Guericke University, Germany
SRI RAJA, TEJASWINI RAJ, Otto von Guericke University, Germany

Recent advances in Retrieval-Augmented Generation (RAG) have shown promising results for medical question answering, yet most benchmark systems evaluated on the MedQA dataset assume answer options are available at inference time — a condition rarely met in real-world clinical settings. In this work, we systematically investigate key design choices in building a RAG-based medical question answering system. We first compare widely used chunking strategies and find that Adaptive Semantic Chunking (ASC) outperforms fixed-size and standard semantic chunking by producing contextually meaningful segments with minimal hyperparameter tuning. The resulting chunks are indexed from medical textbooks and stored in a vector database for efficient retrieval. Building on this, we adopt a multi-agent architecture in which specialized agents decompose questions into subqueries, retrieve the most relevant evidence chunks from the vector database, and synthesize answers by citing supporting evidence, offering pros-and-cons reasoning to enhance explainability. We evaluate the system on the MedQA benchmark under two inference conditions: with and without answer options. When options are provided, the multi-agent system achieves 73.2% accuracy using GPT-OSS-20B, approximately a 10% improvement over the single-agent baseline. In the open-ended setting without options, however, the single-agent configuration achieves higher overall accuracy, yet the two systems exhibit complementary strengths — the multi-agent architecture correctly predicts cases where the single-agent fails, and vice versa. An ensemble combining both outputs leverages these complementary behaviors to achieve the best overall performance of 69.1%, suggesting that multi-agent decomposition does not universally outperform simpler approaches but remains a valuable component in ensemble settings. Finally, faithfulness evaluation using an LLM-as-a-judge metric yields a median score of 0.6, indicating that while most generated claims are grounded in retrieved evidence, hallucination remains a persistent challenge in LLM-based RAG systems.

1 Introduction

Medical question answering requires complex reasoning over specialized domain knowledge. Datasets such as MedQA [8], derived from medical licensing exams, serve as rigorous benchmarks for evaluating language model performance in clinical scenarios.

Standard systems typically predict answers directly from given options. However, in real-world settings options are rarely available, requiring models to demonstrate genuine reasoning rather than option-driven pattern matching. This highlights the need for systems that are both accurate and explainable.

Retrieval-Augmented Generation (RAG) has emerged as a promising approach for knowledge-intensive tasks, grounding model answers in retrieved evidence. However, RAG performance is highly sensitive to chunking strategy and context utilization.

In this work, we investigate explainable medical question answering by combining RAG with a multi-agent architecture. Our key contributions are:

- **Chunking strategy analysis:** We compare multiple chunking methods and adopt a graph-based Adaptive Semantic Chunking (ASC) approach that requires minimal hyperparameter tuning.
- **Multi-agent vs. single-agent comparison:** We systematically compare both architectures, evaluating whether decomposing reasoning across specialized agents yields measurable gains.
- **With-options vs. without-options inference:** We evaluate both conditions to assess whether the system reasons from knowledge or option-level cues.
- **Faithfulness evaluation:** Beyond accuracy, we assess how faithfully generated reasoning is grounded in retrieved evidence.

2 Related Work

Medical question answering has been an active area of research, with datasets like MedQA [8] widely used to test how well models handle clinical reasoning. Early approaches fine-tuned domain-specific transformers such as PubMedBERT [5] and BioBERT [13] on medical multiple-choice exam datasets. However, these models often appeared to rely on superficial textual cues and dataset-specific patterns rather than demonstrating robust medical reasoning or genuine clinical understanding.

As these limitations became apparent, RAG [4] became the go-to approach for grounding model outputs in external knowledge. Several works have since explored how to make RAG work better for medical questions. Adaptive-RAG [7] observes that not every question needs the same retrieval strategy, and trains a small classifier to route each query to direct answering, single-step retrieval, or multi-step iterative retrieval depending on its complexity. i-MedRAG [23] instead keeps retrieval open-ended: the model generates follow-up subqueries one at a time, with each new query shaped by what was already retrieved, gradually building up the evidence needed to answer the original question.

On the reasoning side, MedCoT-RAG [21] shows that structuring the LLM's output around clinical reasoning stages - symptoms, mechanisms, differential diagnoses — leads to more accurate and interpretable answers. RAG2 [18] focuses on what gets passed to the LLM: it uses the model's own reasoning trace as the retrieval query, and then filters the retrieved chunks with a trained model to remove ones that are unlikely to help.

On the chunking side, several works have looked at whether more complex methods are actually worth the extra cost. Renyi et al. [16] compared fixed-size, recursive, and semantic chunking and found that fixed-size chunking often performs just as well or better, despite being much simpler. Mix-of-Granularity [25] tries to get the best of both worlds by training a router that picks between coarse and fine-grained chunks per query, but this requires a separate training phase which is not always practical. RAPTOR [17] takes a more radical approach by replacing flat chunks with a hierarchical tree of recursively summarized text, allowing retrieval at different levels of abstraction. The downside is that building this tree requires many LLM calls during indexing, making it expensive. All three methods highlight the same trade-off: better chunking tends to mean higher cost. To address this, we adopt the ASC method [3] with slight modifications, which avoids both training and LLM calls during indexing; further details are provided in Section 4.1.

Our system takes ideas from each of these works and combines them in a multi-agent setup. Like i-MedRAG, we decompose the question into subqueries before retrieval, but do this in one shot rather than iteratively. Like RAG2, we filter retrieved chunks after retrieval, but use the pretrained MedCPT model [9] instead of training a separate filtering network. And like MedCoT-RAG, we structure reasoning through clinical stages, but distribute the work across specialized agents rather than encoding everything into a single prompt. Finally, while all prior methods evaluate using only answer accuracy, we also measure faithfulness to check whether the model's reasoning is actually supported by the retrieved evidence.

3 Background

This section provides a brief overview of the foundational concepts and components required to understand and build a multi-agent RAG system.

3.1 RAG [4]

RAG augments a generative language model with an external retrieval mechanism, allowing it to ground its responses in retrieved domain-specific evidence rather than relying on knowledge encoded during pretraining. As illustrated in Figure 1, RAG systems can be broadly categorized into three paradigms: Naive RAG, Advanced RAG, and Modular RAG, each representing an increasing level of complexity and flexibility.

3.1.1 Types of RAG.

(1) **Naive RAG**: The simplest form of RAG, consisting of three sequential steps: *Indexing*, in which documents are converted into vector representations; *Retrieval*, in which the most relevant chunks are fetched; and *Generation*, in which the retrieved chunks are passed to an LLM to synthesize a final answer.
(2) **Advanced RAG**: An extension of Naive RAG designed to address its limitations through two additional stages. In the *Pre-Retrieval* stage, techniques such as query expansion or query rewriting are applied to improve retrieval quality. In the *Post-Retrieval* stage, a reranking model evaluates the retrieved chunks to ensure only the most relevant context is forwarded to the LLM, and context compression is applied to remove irrelevant or redundant content.
(3) **Modular RAG**: The most advanced and flexible RAG paradigm. Rather than following a fixed pipeline, it treats the system as a collection of independent, specialized modules, each of which can be implemented as a dedicated agent.

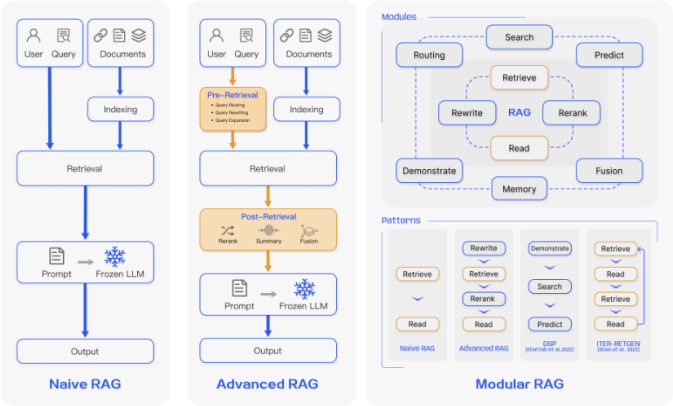


Fig. 1. Taxonomy of RAG [4]

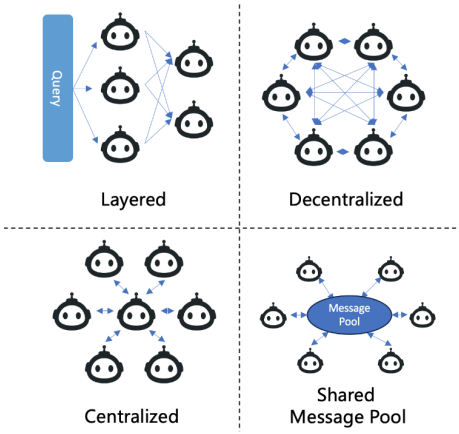


Fig. 2. Communication Topologies in Multi-Agent LLM Systems [6]

3.2 Multi-Agent Systems [6]

3.2.1 Communication Structures.

(1) **Layered**: Agents are organized in a hierarchical structure in which each level is assigned specific roles. Interactions occur primarily within a layer or between adjacent layers, enabling structured and efficient coordination (Figure 2).
(2) **Decentralized**: Agents communicate in a peer-to-peer fashion, interacting directly with one another without relying on a central coordinating node.
(3) **Centralized**: One or more central agents manage and coordinate all interactions within the system. Other agents communicate primarily through this central node, which controls information flow and task assignment.
(4) **Shared Message Pool**: Rather than communicating directly, agents publish messages to a common shared space. Each agent reads only the information relevant to its role, reducing unnecessary communication and improving coordination efficiency.

3.3 Chunking Methods

Chunking is the process of splitting large texts into smaller, meaningful segments to remain within the context window of LLMs and to improve the relevance of retrieved information in RAG systems. The following methods represent the most commonly adopted chunking strategies in RAG-based systems.

(1) **Fixed-Size**: Text is divided into segments of a fixed number of tokens or characters, often with an overlap parameter to preserve local context across boundaries. While computationally efficient and straightforward to implement, this method frequently disregards structural and semantic document boundaries, potentially splitting critical information across segments [6, 10].
(2) **Recursive**: This approach applies a hierarchical set of separators — such as double newlines, single newlines, and spaces — to divide text. It prioritizes keeping related content together by falling back to finer delimiters only when a segment exceeds the defined chunk size and overlap constraints, resulting in a more structure-aware segmentation than naive fixed-size splitting [12].
(3) **Semantic**: Sentence embeddings are used to detect thematic shifts by computing the cosine similarity between adjacent sentence groups, where a *buffer size* parameter determines how many surrounding sentences are grouped together when evaluating similarity. Chunk boundaries are identified where similarity drops significantly, based on a defined breakpoint threshold that can be determined using percentile-based, standard deviation-based, or interquartile range (IQR) strategies, ensuring that each resulting chunk is contextually coherent [11].

3.4 Vector Databases

A key component of RAG systems is the storage of processed text chunks in a vector database. After chunking, each segment is converted into a dense vector representation using a neural embedding model, then indexed for efficient similarity-based retrieval at query time. Numerous vector databases have been developed, including Chroma [2], Weaviate [22], Pinecone [14], and Qdrant [15]; many other systems are discussed in [20]. These systems are optimized for high-dimensional similarity search using metrics such as cosine similarity.

4 Method

In this section, we describe the proposed methodology for document chunking and the overall model architecture in detail.

4.1 Adaptive Semantic Chunking Method (ASCM)

As discussed earlier, chunking plays a critical role in RAG systems, and the final performance is highly sensitive to the quality of the generated chunks. In this work, we evaluate and compare three different chunking strategies introduced in Section 3.3. The corresponding experiments and results are reported in Section 7. Here, we focus on the proposed modification to existing semantic chunking approaches, inspired by Fartushny et al. [3].

A key challenge in semantic chunking is the selection of hyperparameters, particularly the buffer size and similarity threshold. Increasing the buffer size improves contextual continuity but leads to higher computational cost. Conversely, smaller buffer sizes reduce computational overhead but may fail to capture broader semantic dependencies, resulting in fragmented and incoherent chunks [24]. Similarly, the choice of thresholding strategy significantly affects chunk quality, and selecting optimal values is non-trivial due to dataset-dependent variability.

To address these limitations, we adopt and extend the graph-based semantic chunking framework proposed by Fartushny et al. [3], thereby reducing the number of required hyperparameters to only two: the maximum token length $M_{\max}$ and the minimum token length $M_{\min}$.

At a high level, ASCM treats document segmentation as an optimization problem. Rather than splitting text at fixed intervals or predefined thresholds, it models the document as a sequence of sentences and searches for

the segmentation that minimizes a global cost function. This cost function balances three competing objectives: placing boundaries at semantically weak points between sentences, keeping chunks close to an optimal length, and ensuring that sentences within each chunk are semantically consistent with one another. The optimal segmentation is found efficiently using dynamic programming, avoiding the need to exhaustively evaluate all possible splits. The following paragraphs formalize each of these components mathematically.

The method models a document as a graph $G = (V, E)$, where each node $v_i \in V$ corresponds to a sentence, and edges $e_{ij} \in E$ represent relationships between sentences. Let $j \in \{1, 2, \ldots, n\}$ denote the chunk ending index. For each j, the valid starting index i is constrained by the chunk length bounds $i \in [\max(0, j - L_{\max}(j)),\ j - L_{\min}(j)]$. The quantities $L_{\min}$ and $L_{\max}$ are defined later in Equations 10 and 9, respectively.

The edge cost $\phi(i, j)$ evaluates the quality of forming a chunk from sentence s_i to s_j and is defined as:

$$\phi(i, j) = \alpha\, C_{\text{boundary}}(j) + \lambda\, P_{\text{len}}(i, j) + \mu\, P_{\text{coh}}(i, j) \tag{1}$$

where $C_{\text{boundary}}(j)$ captures boundary strength between adjacent sentences, $P_{\text{len}}(i, j)$ penalizes deviations from an optimal chunk length, and $P_{\text{coh}}(i, j)$ enforces internal coherence within a chunk. The parameters α, λ, and μ control the relative importance of each term.

The boundary cost $C_{\text{boundary}}(j)$ quantifies the desirability of placing a chunk boundary *after* sentence j:

$$C_{\text{boundary}}(j) = \frac{1}{1 + C_{\text{total}}(j)} \tag{2}$$

where $C_{\text{total}}(j)$ is a composite measure of semantic dissimilarity between consecutive sentences s_j and s_{j+1}:

$$C_{\text{total}}(j) = \delta\, C_{\text{sem}}(s_{j-1}, s_j) + \beta\, C_{\text{struct}}(s_{j-1}, s_j) + \gamma\, C_{\text{med}}(s_{j-1}, s_j) \tag{3}$$

Here, C_{sem} represents semantic dissimilarity based on cosine distance between sentence embeddings, C_{struct} encodes structural information (e.g., section or paragraph boundaries), and C_{med} captures domain-specific constraints. The parameters δ, β, and γ control the contribution of each component.

In practice, both C_{struct} and C_{med} are difficult to define consistently across heterogeneous documents, as document structure and domain-specific constraints vary significantly. Therefore, we simplify the formulation by removing manually engineered structural and constraint-based components.

Instead, we approximate structural boundaries using paragraph segmentation, where text is initially split at newline markers. Each paragraph is treated as a candidate chunk unit. However, this introduces two edge cases: (i) overly long paragraphs, and (ii) very short paragraphs (e.g., headings). To address this, long paragraphs are further segmented into sentences, and the ASC algorithm 1 is applied at the sentence level. Conversely, short paragraphs are merged until a minimum contextual length is reached before applying the chunking procedure.

Under this simplified formulation, the total cost reduces to a purely semantic similarity-based measure:

$$C_{\text{total}}(j) = \cos\left(\mathbf{s_{j-1}}, \mathbf{s_j}\right) \tag{4}$$

To compute P_{len}, we first need to define the optimal chunk length L_{opt}. In the original formulation, this is treated as a fixed hyperparameter. However, we found this difficult to justify in practice, as the appropriate chunk length varies significantly across documents with different sentence densities and lengths. To address this, we instead compute L_{opt} dynamically for each position i, based on a maximum token budget $M_{\max}$. Let w_t denote the number of words in sentence t. The optimal length $L_{\text{opt}}(i)$ starting from position i is then defined as the longest span whose total word count does not exceed $M_{\max}$:

$$L_{\text{opt}}(i) = \max\left\{ \ell \in \mathbb{N} \;\middle|\; \sum_{t=i}^{i+\ell-1} w_t \le M_{max} \right\} \tag{5}$$

Using this optimal length, we define the actual span of a candidate chunk between sentence indices i and j as:

$$L(i, j) = j - i \tag{6}$$

The length-based penalty term P_{len} is then formulated as a normalized quadratic deviation from the optimal chunk length:

$$P_{\text{len}}(i, j) = \frac{\left(L(i, j) - L_{\text{opt}}(i)\right)^2}{L_{\text{opt}}(i)} \tag{7}$$

This penalty discourages deviations from the optimal chunk size by penalizing both excessively short and overly long chunks, while the normalization by $L_{\text{opt}}(i)$ ensures that the penalty remains scale-invariant across documents with varying sentence densities and lengths.

The second key component of the cost function is the coherence penalty $P_{\text{coh}}(i, j)$ defined in Equation 1. This term is designed to quantify the internal semantic consistency of a candidate chunk by measuring the average dissimilarity between consecutive sentences within the span.

$$P_{\text{coh}}(i, j) = \begin{cases} 0, & \text{if } j - i \leq 1 \\ \dfrac{1}{(j-1) - i} \displaystyle\sum_{k=i}^{j-2} d_k, & \text{otherwise} \end{cases} \tag{8}$$

Here, d_k represents the semantic distance between adjacent sentences s_k and s_{k+1}, typically computed as the complement of cosine similarity in the embedding space. The averaging term normalizes the coherence score with respect to chunk length, ensuring that longer chunks are not unfairly penalized due to having more sentence transitions.

This formulation encourages the construction of chunks that are not only within an appropriate length range but also exhibit smooth and consistent semantic progression throughout their content. Consequently, chunks with abrupt topic shifts or weak inter-sentence relationships receive higher coherence penalties, making them less likely to be selected by the optimization procedure.

In Algorithm 1, an important component is the definition of the lower and upper bounds on chunk length, denoted as $L_{\min}(j)$ and $L_{\max}(j)$. These bounds ensure that generated chunks are neither excessively long nor too short, enforcing the constraints:

$$j - i \leq L_{\max}(j) \quad \text{(not too long)}, \qquad j - i \geq L_{\min}(j) \quad \text{(not too short)}.$$

As highlighted in prior work, these parameters are typically assumed to be fixed. However, in our setting they are difficult to predefine due to variability in document structure and length. To address this limitation, we compute them dynamically for each endpoint position j.

Specifically, we define the set of feasible chunk lengths ending at position j as:

$$L_{\max}(j) = \max \left\{ k \in \mathbb{N} \;\middle|\; \sum_{t=j-k+1}^{j} w_t \leq M_{max} \right\} \tag{9}$$

$$L_{\min}(j) = \min \left\{ k \in \mathbb{N} \;\middle|\; \sum_{t=j-k+1}^{j} w_t \geq M_{min} \right\} \tag{10}$$

After applying Algorithm 1, the optimal chunks are recovered through a simple backtracking step. During the forward pass, the algorithm fills a `parent` array, where each entry records the best split point for the chunk ending at that sentence. Once the forward pass is complete, recovering the final segmentation is straightforward:

we start from the last sentence and repeatedly follow the `parent` pointers backwards until we reach the beginning. Each pointer we follow defines one chunk. Figure 3 shows an example of this process.

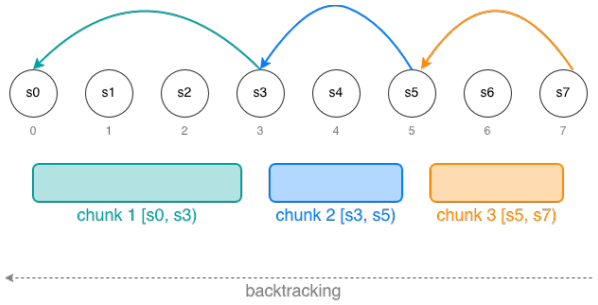


Fig. 3. Illustration of the backtracking procedure in ASCM

Algorithm 1 ASCM Chunking via Dynamic Programming

Require: Sentences $S = \{s_0, \ldots, s_{n-1}\}$
Ensure: Optimal segmentation of sentences into chunks

```
n ← |S|
dp[0 ... n] ← +∞,    dp[0] ← 0
parent[0 ... n] ← −1
Compute optimal chunk size parameter via L_opt
for j = 1 to n do
    Determine (L_min, L_max) for position j
    for i = max(0, j − L_max) to j − L_min do
        cost ← dp[i] + φ(i, j)
        if cost < dp[j] then
            dp[j] ← cost
            parent[j] ← i
        end if
    end for
end for
return parent {Backtrack to recover optimal chunks} =0
```

In our experiments, we used the MedEmbed-large-v0.1 model [1] to generate embeddings, as it has been shown to outperform other models in medical text retrieval tasks. After applying the aforementioned algorithm, text chunks are generated and stored in a ChromaDB vector database, which is subsequently used for retrieval during inference.

4.2 RAG Architecture

4.2.1 Simple RAG / Naive RAG. We consider this model architecture as a baseline and divide it into two variants. In the first variant, the model is provided with both the question and the answer options during response generation. In the second variant, only the question is given to the model, which generates an answer independently. The

generated answer is passed to a second LLM along with the answer choices, which then selects the correct option for use in later model evaluation.

The basic architecture (Fig 4) is as follows: the input (question, with or without options) is first embedded using the MedEmbed model. Relevant chunks are then retrieved based on these embeddings, and the retrieved context is passed to the LLM to generate the final answer. When options are not provided to the LLM, it generates an answer using only the information from the chunks and produces a free-form response. This answer is then passed to the Option Matching LLM to check whether it matches any of the given options.

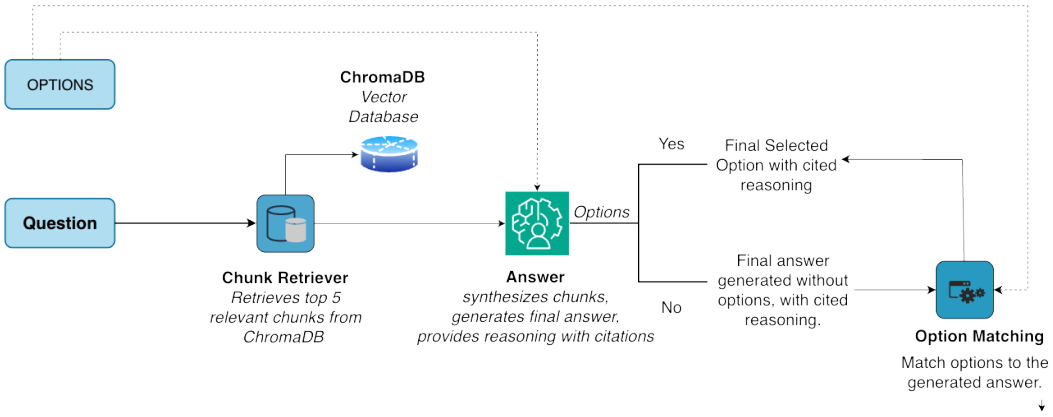


Fig. 4. Single-Agent RAG Pipeline

4.2.2 Multi-Agent. This is the modular RAG architecture discussed in Section 3.1.1. The overall model architecture is illustrated in Figure 5. The design of this multi-agent system is inspired by how a human approaches complex reasoning tasks. Similar to the Naive RAG setup, this model operates in two modes: with answer options and without answer options. When options are available, they are provided to the *AnswerAgent* along with the original question and evidences to guide the final selection. In the absence of options, the agent generates a free-form answer based solely on the synthesized insights. This generated answer is then passed to an *OptionMatchingAgent*, which receives both the generated answer and the original answer choices, and selects the option that best aligns with the predicted answer.

The first component, *DomainExtractorAgent*, identifies the domain from which the question originates. This allows subsequent agents to constrain their reasoning and retrieval to domain-relevant knowledge.

The next component, *VignetteAgent*, extracts key contextual details from the question, such as age, sex, important findings, and the core problem. We introduced this agent because directly passing the full question to an LLM often leads to important details being overlooked, which can negatively impact answer accuracy.

Following this, the *SubqueryBuilderAgent* generates multiple subqueries using the extracted domain, question options (when provided) and vignette information. Typically, this agent produces 3–4 subquestions derived from the original query. Each subquestion is designed to focus on a single medical concept required to answer the main question and is fully self-contained. When options are available, they are also provided to this agent so that the subqueries generated by the LLM can use information from the options.

After generating the subqueries, they are passed to a *ChunkRetrieverAgent*, which retrieves the top 10 relevant document chunks from the ChromaDB vector database. These retrieved chunks are then forwarded to a ranking and filtering component that leverages MedCPT [9].

MedCPT is a pretrained transformer-based architecture trained using contrastive learning on Medical data, designed specifically for medical document retrieval. In our pipeline, each query–document pair (i.e., the question and a retrieved chunk) is passed individually to the MedCPT model, which outputs a relevance score. Based on these scores, the initially retrieved top 10 chunks are re-ranked, and the top 5 most relevant documents are selected.

In parallel, the *SubqueryAnswerAgent* processes each subquery independently. Using the filtered context from the retrieved chunks, it generates candidate answer for each subquery. The agent also cites the specific chunks used as evidence and provides reasoning to support generated answer.

Finally, the *AnswerAgent* generates the answer to the original question by aggregating information from all subqueries. It takes as input the subquery questions, their corresponding answers, supporting evidence, options (when provided) and reasoning. The agent synthesizes these insights in a step-by-step manner to arrive at the final answer, while also providing reasoning supported by cited evidence. This agent is designed to return only the correct answer option when the supporting insights provide sufficient evidence. If the evidence does not support any option, it returns Option X, indicating that the answer is not supported by the available evidence.

As described earlier, to improve explainability, retrieved chunks are assigned unique identifiers and passed to the LLM. This enables the agents to reference specific evidence by citing chunk IDs in their reasoning that evaluates answer choices using a pros-and-cons analysis, with each argument supported by cited chunk IDs.

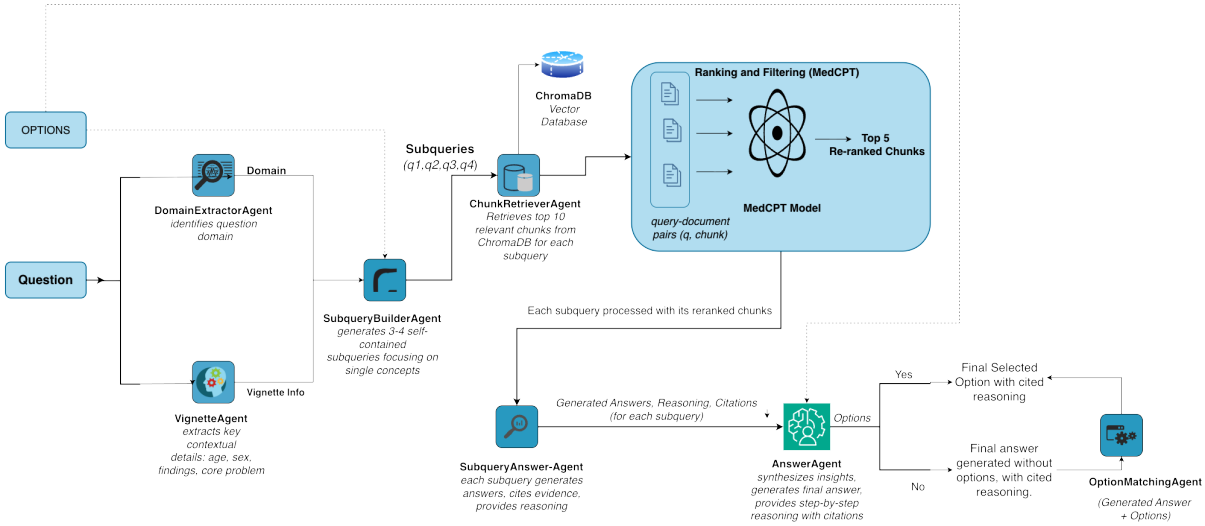


Fig. 5. Multi-Agent RAG Architecture

5 Evaluation Framework

This section contains a brief overview of the two metrics we used to evaluate our model. To assess the model's performance, we used Accuracy. Our system enhances explainability by having agents cite supporting evidence chunks in their reasoning. To quantify how faithfully the generated answers are grounded in this evidence, we employ a faithfulness metric as described below.

5.1 Answer Accuracy

Answer Accuracy is a standard evaluation metric used to compare the performance of different models. It measures the proportion of questions for which the model selects the correct final answer.

Formally, it is defined as:

$$\text{Answer Accuracy} = \frac{\text{Number of Correctly Answered Questions}}{\text{Total Number of Questions}} \tag{11}$$

5.2 LLM-as-a-judge Evaluation

We evaluate the proposed system using an LLM-based faithfulness metric, inspired by the RAGAs framework [19]. This metric assesses whether the generated answer is supported by the retrieved evidence, ensuring consistent evaluation of semantic correctness and contextual grounding across samples.

5.2.1 Faithfulness Evaluation. Faithfulness measures the extent to which the generated answer is grounded in the provided contextual evidence. First, the final model output is decomposed into a set of atomic factual claims using an LLM-based claim extraction prompt, where each claim represents a single, verifiable statement.

For each extracted claim, we construct a context consisting of all subquery-level insights, where each insight includes the subquery question, its answer, reasoning, and supporting evidence chunks generated by the *SubqueryAnswerAgent* as discussed in Section 4.2.2. An LLM evaluator then determines whether each claim is explicitly supported by the provided context, producing a binary decision (YES/NO). The final faithfulness score is computed as:

$$\text{Faithfulness} = \frac{\text{Number of Supported Claims}}{\text{Total Number of Claims}} \tag{12}$$

6 Experimental Setup

In this section, we describe the experimental design and parameter settings used throughout our study. All experiments are conducted on the MedQA dataset [8], which contains 1,273 multiple-choice questions. The dataset also provides access to relevant textbooks, which we utilize as contextual sources in our Retrieval-Augmented Generation (RAG) system.

6.1 Chunking Method Performance

We evaluate and compare multiple chunking strategies described in Section 3.3, including semantic chunking and fixed-size chunking, as well as ASCM (Section 4.1). For semantic chunking, we set the buffer size to 2 and employ a standard deviation-based thresholding method. Additionally, we analyze how variations in the standard deviation parameter influence chunk breakpoints.

For fixed-size chunking, the chunk size is set to 150 tokens. In the case of ASCM, the minimum and maximum token limits are set to 50 and 150, respectively.

All chunking methods are evaluated using the Simple RAG framework (Section 4.2.1), with a retrieval size of 5 chunks. We use the LLaMA-3.1-8B-Instruct model with the temperature parameter fixed at 0 to ensure deterministic outputs.

6.2 Single-Agent vs. Multi-Agent

After identifying the best-performing chunking strategy, we use it to compare Single-Agent (Section 4.2.1) and Multi-Agent (Section 4.2.2) RAG architectures. To evaluate the effect of model scale, we conduct experiments using two language models: LLaMA-3.1-8B-Instruct, representing a smaller model, and OpenAI GPT-OSS-20B,

representing a larger reasoning-oriented model. Due to computational constraints, our study is limited to these two models.

For both models, the temperature is set to 0 to maintain consistency and reproducibility across experiments.

6.3 Without-Options Inference

We further investigate the impact of providing answer options during inference. Specifically, we compare model performance when answer choices are included versus when the model must generate answers without options.

This experiment is designed to analyze whether the model relies on pattern matching among the provided options or demonstrates genuine reasoning capabilities. Additionally, we examine whether the model can correctly generate answers independently, effectively ignoring distractor choices when they are present.

6.4 Faithfulness Evaluation

We evaluate whether the answers generated by the *AnswerAgent* are grounded in the subquery-level insights provided to it, or whether the model produces unsupported claims. Due to computational and time constraints, this evaluation is conducted exclusively for the GPT-OSS-20B multi-agent configuration.

7 Results

7.1 Chunking Method Performance

As shown in Table 1, the ASCM method performs best among all the chunking methods we tested. Changing only the chunking method itself leads to improvements in the results. One advantage of this method is that it does not require many hyperparameters, except for the minimum and maximum chunk sizes.

The second-best method is fixed-size chunking, while the worst-performing method is semantic chunking. The main issue with semantic chunking is that it has too many parameters, which are difficult to set optimally. As shown in Figure 6, when using standard deviation as the threshold calculation, different values of k in $\text{mean} + k \cdot \text{std}$ result in different numbers of chunks. This makes it very difficult to determine the best value, as different values of k produce different breakpoints and chunk boundaries.

Method	**Chunk Used**	**Accuracy (%)**	**Correct / Total**
FixedSize	5	56.48	719 / 1273
Semantic Chunking	5	56.09	714 / 1273
ASCM	5	**57.42**	**731 / 1273**

Table 1. Comparison of chunking methods. Best results are shown in bold.

7.2 Single-Agent vs. Multi-Agent

Table 2 shows that the OpenAI GPT-OSS-20B multi-agent system achieves the highest overall accuracy of 73.2%, outperforming all other evaluated models, including the Naive RAG baseline. This indicates that multi-agent architectures can significantly improve performance when built on top of strong reasoning models.

However, this improvement is not consistent across all model scales. For smaller models with limited reasoning capacity, the multi-agent setup does not necessarily yield gains. In this case, the multi-agent configuration achieves an accuracy of 57.8%, which is slightly lower than the single-agent performance of 58.6%.

In contrast, when using the larger GPT-OSS-20B reasoning model, the multi-agent approach improves performance by approximately 10% compared to the single-agent baseline. These results suggest that the effectiveness of multi-agent systems is strongly dependent on the underlying model's reasoning capabilities.

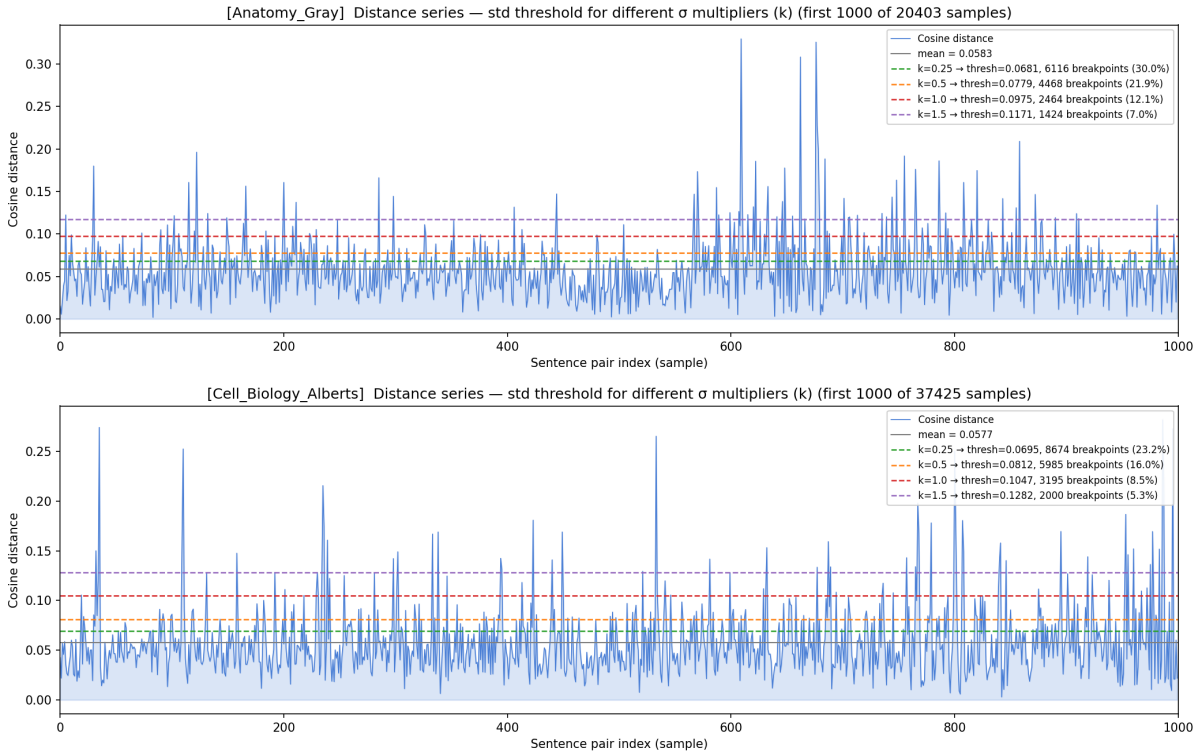


Fig. 6. Effect of Threshold Parameter on Semantic Breakpoint Detection

Setup	Model	Accuracy (%)	Correct / Total
SingleAgent	openai/gpt-oss-20b	63.08	803 / 1273
	llama-3.1-8b-instant	58.6	746 / 1273
Multiagent	llama-3.1-8b-instant	57.8	736 / 1273
	openai/gpt-oss-20b	**73.2**	**932 / 1273**

Table 2. Performance comparison between SingleAgent and Multiagent models. Best results are in bold.

7.3 Without-Options Inference

In the case without options, the results were completely opposite, as shown in Table 3. For the Multi-agent setup, when options were not provided to the LLM, the Single Agent LLaMA model underperformed compared to the Multiagent Agent LLaMA model, dropping from 46.1% (Multi-agent) to 43.36% (Single Agent). In the case of the GPT-OSS-20B model, the Single Agent configuration achieved the highest performance at 60.8%, which was slightly reduced in the Multi-agent setting. This suggests that providing options during subquery generation plays a significant role in improving Multi-agent performance.

To better understand this effect, Figure 9a illustrates the answer distribution for both configurations when options are absent. Both systems answered 586 questions correctly and got 345 questions wrong. The remaining

questions were split almost evenly — 154 were answered correctly only by Multi-agent, and 188 were answered correctly only by Single Agent. This indicates a complementary behavior: samples that were incorrectly predicted by the Single Agent were correctly answered by the Multi-agent, and vice versa. This clearly demonstrates that sub-queries assisted the Multi-agent in arriving at correct answers in cases where the Single Agent failed, while in other cases the Single Agent performed better and the sub-queries did not contribute additional benefit. This behavior, however, was not observed when options were provided. As shown in Figure 9b, the Multi-agent configuration correctly predicted approximately double the number of samples compared to the Single Agent alone, highlighting the substantial benefit of incorporating options during query generation.

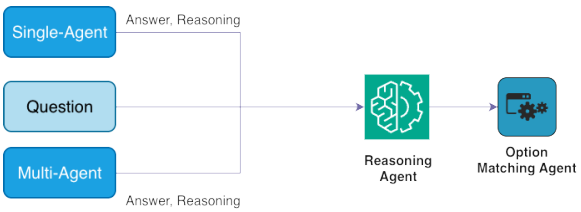


Fig. 7. Ensemble Reasoning Pipeline for Open-Ended Inference

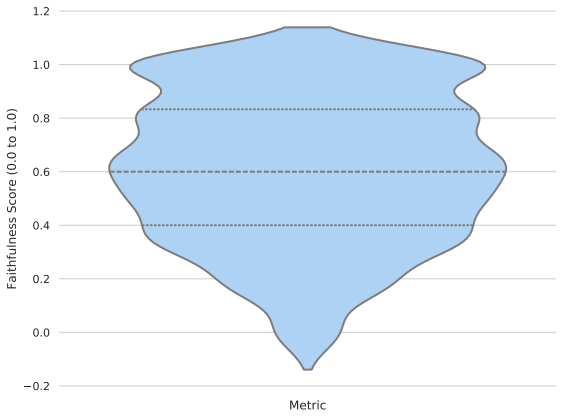
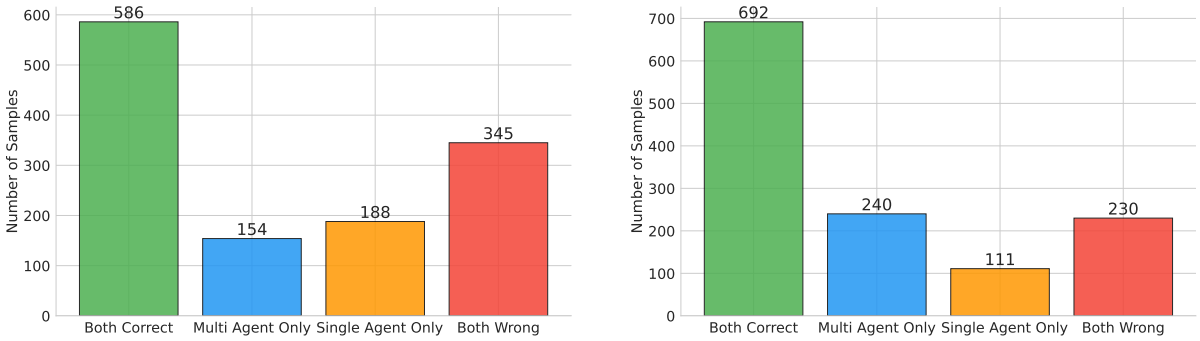


Fig. 8. Distribution of Faithfulness Scores

To further improve performance, we combined the answers generated by both the Multi-agent and Single Agent systems. Specifically, we collected the question, the generated answer, and the reasoning provided by each agent. A dedicated Reasoning Agent was then deployed to evaluate which answer best fits the question based on the provided reasoning from each agent (Fig 7). As shown in Table 3, the ensemble method outperforms all other configurations.



(a) Without options

(b) With options

Fig. 9. Comparison of multi-agent and single-agent performance: (a) without options; (b) with options.

Setup	Model	Accuracy (%)	Correct / Total
SingleAgent	openai/gpt-oss-20b	60.8	774 / 1273
	llama-3.1-8b-instant	43.36	552 / 1273
Multiagent	openai/gpt-oss-20b	58.1	740 / 1273
	llama-3.1-8b-instant	46.1	587 / 1273
Ensemble	openai/gpt-oss-20b	**69.13**	**880 / 1273**

Table 3. Performance comparison of SingleAgent and Multiagent models in open-ended (no options) setting. Best results are shown in bold.

Figure 8 shows the distribution of faithfulness scores across the evaluated samples. The median faithfulness score of approximately 0.6 indicates that, on average, 60% of the claims in the generated answers are grounded in the retrieved evidence. The distribution is notably dense between 0.4 and 0.83, suggesting that the majority of samples achieve moderate to high faithfulness. However, a small number of samples exhibit near-zero faithfulness, indicating cases where the *AnswerAgent* produced claims not supported by the provided context, indicating that the LLM hallucinated.

8 Conclusion

In this work, we investigated two complementary dimensions of RAG system design for medical question answering: document chunking and multi-agent architecture. We examined the limitations of existing chunking approaches and adopted the graph-based ASC method [3] with slight modifications, which dynamically computes chunk boundaries through a cost-aware dynamic programming formulation requiring minimal hyperparameter tuning. Experimentally, ASC outperformed both fixed-size and standard semantic chunking on the MedQA benchmark, albeit with a modest margin.

Beyond chunking, we demonstrated that a structured multi-agent architecture can substantially improve answer quality, but only when the underlying model has sufficient reasoning capacity. For smaller models, the added complexity of multi-agent coordination provided no benefit. Furthermore, we observed that decomposing questions into subqueries does not always improve performance — in some cases, directly passing the original question to the model yielded better results, suggesting that subquery generation can sometimes introduce unnecessary complexity or loss of context.

Finally, our ensemble approach, combining single-agent and multi-agent outputs through a dedicated reasoning agent, achieved the strongest results in the open-ended setting, suggesting the two architectures exhibit complementary strengths.

Faithfulness evaluation revealed that despite explicitly prompting the *AnswerAgent* to ground its responses in the provided context, the median faithfulness score of approximately 0.6 indicates that a notable portion of generated claims remain unsupported by the retrieved evidence. This suggests that the model occasionally relies on its parametric knowledge rather than the retrieved passages, highlighting a known limitation of LLM-based RAG systems.

We believe that performance could be further improved by expanding the retrieval corpus beyond textbook material to include a broader range of medical literature which may provide richer and more diverse evidence for complex medical questions.

References

[1] Abhinand Balachandran. 2024. *MedEmbed: Medical-Focused Embedding Models.* https://github.com/abhinand5/MedEmbed

[2] chroma-core. 2025. Chroma: The AI-native open-source embedding database. https://github.com/chroma-core/chroma Accessed: 2025-02-21.
[3] E Fartushny, G Lebedev, and V Kuzovatova. 2026. Adaptive Semantic Chunking Method for Medical Retrieval-Augmented Generation Systems. (2026).
[4] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023), 32.
[5] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare (HEALTH)* 3, 1 (2021), 1–23.
[6] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).
[7] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 7036–7050.
[8] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences* 11, 14 (2021), 6421.
[9] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. 2023. MedCPT: Contrastive Pre-trained Transformers with large-scale PubMed search logs for zero-shot biomedical information retrieval. *Bioinformatics* 39, 11 (2023), btad651.
[10] LangChain AI. 2024. CharacterTextSplitter: API Reference. https://reference.langchain.com/python/langchain-text-splitters/character/CharacterTextSplitter. Accessed: 2026-04-17.
[11] LangChain AI. 2024. langchain-experimental: text_splitter.py. https://github.com/langchain-ai/langchain-experimental/blob/main/libs/experimental/langchain_experimental/text_splitter.py. Accessed: 2026-04-17.
[12] LangChain Inc. 2024. RecursiveCharacterTextSplitter | LangChain Python API Reference. https://reference.langchain.com/python/langchain-text-splitters/character/RecursiveCharacterTextSplitter Accessed: 2024-05-22.
[13] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* 36, 4 (2020), 1234–1240.
[14] Pinecone Systems Inc. 2026. Pinecone: The vector database to build knowledgeable AI. https://www.pinecone.io/ Accessed: 2026-04-17.
[15] Qdrant. 2025. Qdrant: High-performance, massive-scale Vector Database and Vector Search Engine for the next generation of AI. https://github.com/qdrant/qdrant Accessed: 2025-02-21.
[16] Renyi Qu, Ruixuan Tu, and Forrest Bao. 2025. Is semantic chunking worth the computational cost?. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 2155–2177.
[17] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
[18] Jiwoong Sohn, Yein Park, Chanwoong Yoon, Sihyeon Park, Hyeon Hwang, Mujeen Sung, Hyunjae Kim, and Jaewoo Kang. 2025. Rationale-guided retrieval augmented generation for medical question answering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 12739–12753.
[19] VibrantLabs. 2024. Ragas: Supercharge Your LLM Application Evaluations. https://github.com/vibrantlabsai/ragas.
[20] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2025. Towards reliable vector database management systems: A software testing roadmap for 2030. *arXiv preprint arXiv:2502.20812* (2025).
[21] Ziyu Wang, Elahe Khatibi, and Amir M Rahmani. 2025. MedCoT-RAG: Causal Chain-of-Thought RAG for Medical Question Answering. In *2025 IEEE 21st International Conference on Body Sensor Networks (BSN)*. IEEE, 1–4.
[22] Weaviate. 2025. Weaviate: Open-source Vector Database. https://github.com/weaviate/weaviate Accessed: 2025-02-21.
[23] Guangzhi Xiong, Qiao Jin, Xiao Wang, Minjia Zhang, Zhiyong Lu, and Aidong Zhang. 2024. Improving retrieval-augmented generation in medicine with iterative follow-up questions. In *Biocomputing 2025: Proceedings of the Pacific Symposium*. World Scientific, 199–214.
[24] Kezhen Zhong, Basem Suleiman, Abdelkarim Erradi, and Shijing Chen. 2025. Semrag: Semantic knowledge-augmented rag for improved question-answering. *arXiv preprint arXiv:2507.21110* (2025).
[25] Zijie Zhong, Hanwen Liu, Xiaoya Cui, Xiaofan Zhang, and Zengchang Qin. 2025. Mix-of-granularity: Optimize the chunking granularity for retrieval-augmented generation. In *Proceedings of the 31st International Conference on Computational Linguistics*. 5756–5774.